6CS005
High Performance Computing
Week 10 Workshop

Tasks – CUDA Block, Grid and Thread

Student Number: 2408869

Name: Narayan Lohani

Group: L6CG15

Table of Contents

Question 1: Function Calls Inside CUDA Kernels

The following CUDA program attempts to launch a GPU kernel that calls a function to print **"Hello World"**.

```
#include <stdio.h>

void displayHelloWorld(){
   printf("Hello World\n");
}
__global__ display(){
   displayHelloWorld();
}
int main(){
   display<<<1,1>>>();
   cudaDeviceSynchronize();
   return 0;
}
```

(a) Does the code work?

No, the code does not print “Hello World” instead it throws compile-time error as the displayHelloWorld function isn’t a device function defined with __device__ making its call attempt inside kernel function invalid.

Also, the kernel function defined with __global__ is missing return type which throws another compile-time error.

(b) Modify the code such that the function displayHelloWorld is callable inside the kernel.

To make the code work, we just need to add __device__ in front of the function declaration.

hpc_herald - helloWorld.cu

```
#include <stdio.h>
__device__ void displayHelloWorld() {
    printf("Hello World\n");
}
__global__ void display() {
    displayHelloWorld();
}
int main() {
    display << <1, 1 >> > ();
    cudaDeviceSynchronize();
    return 0;
}
```

```
[8] ✓ 1s   1 !nvcc -arch=sm_75 -o HelloWorld HelloWorld.cu
[9] ✓ 0s   1 !./t
           Hello World
```

Question 2: Thread and Block Indexing

Run and observe the output of the following CUDA program:

```
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void display() {
    int threadID = threadIdx.y;
    int blockID = blockIdx.x;
    printf("Block %d -> ThreadId = %d\n", blockID, threadID);
}
int main() {
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(5, 1, 1);
    display<<<gridSize, blockSize>>>();
    cudaDeviceSynchronize();
    return 0;
}

 Expected Output:
Block 0 -> ThreadId = 0
Block 0 -> ThreadId = 1
Block 0 -> ThreadId = 2
Block 0 -> ThreadId = 3
Block 0 -> ThreadId = 4
```

(a) Does the actual output match the expected output? Identify the problem.

No, the actual output does not match the expected output.

Actual Output:

```
   1 !./Thread_Block_Indexing

Block 0 -> ThreadId = 0
Block 0 -> ThreadId = 0
Block 0 -> ThreadId = 0
Block 0 -> ThreadId = 0
Block 0 -> ThreadId = 0
```

This is because, the code is printing threadIdx.y with refers to the index of thread in second dimension. However, current execution involves a single block contained five threads in single dimension. As the threads are 1D, the values of threadIdx.y are 0 for all threads.

(b) Modify the code so that it produces the correct output and explain your changes.

To produce the correct output, we simple change the threadId to store threadIdx.x instead of threadIdx.y. This change makes the threads to print their index in x-dimension resulting in ThreadId from 0 to 4.

hpc_herald - 2.cu

```cpp
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void display() {
    int threadID = threadIdx.x;
    int blockID = blockIdx.x;
    printf("Block %d -> ThreadId = %d\n", blockID, threadID);
}
int main() {
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(5, 1, 1);
    display << <gridSize, blockSize >> > ();
    cudaDeviceSynchronize();
    return 0;
}
```

[14] ✓ 1s

```
!nvcc -arch=sm_75 -o Thread_Block_Indexing Thread_Block_Indexing.cu
```

[15] ✓ 0s

```
!./Thread_Block_Indexing
```

```
Block 0 -> ThreadId = 0
Block 0 -> ThreadId = 1
Block 0 -> ThreadId = 2
Block 0 -> ThreadId = 3
Block 0 -> ThreadId = 4
```

Question 3: Multi-Dimension Block and Thread Indexing

In the code above, the block dimension is specified as dim3 blockSize(5, 1, 1).
Which means that the block has:

- 5 threads in the x-dimension
- 1 thread in the y-dimension
- 1 thread in the z-dimension

This typically means that the block we created is a 1D block with 5 threads in the x-direction.

a. Modify the code so that it creates a 2D block instead of a 1D block. Also print thread ids along its corresponding directions/axis.

hpc_herald - 3.cu

```
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void display() {
    printf("Block %d → tid X = %d and tid Z = %d\n",
        blockIdx.x, threadIdx.x, threadIdx.z);
}
int main() {
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(5, 1, 3);
    display << <gridSize, blockSize >> > ();
    cudaDeviceSynchronize();
    return 0;
}
```

```
[10] ✓ 1s
1 !nvcc -arch=sm_75 -o 2d_threads 2d_threads.cu

[11] ✓ 0s
1 !./2d_threads

Block 0 -> tid X = 0 and tid Z = 0
Block 0 -> tid X = 1 and tid Z = 0
Block 0 -> tid X = 2 and tid Z = 0
Block 0 -> tid X = 3 and tid Z = 0
Block 0 -> tid X = 4 and tid Z = 0
Block 0 -> tid X = 0 and tid Z = 1
Block 0 -> tid X = 1 and tid Z = 1
Block 0 -> tid X = 2 and tid Z = 1
Block 0 -> tid X = 3 and tid Z = 1
Block 0 -> tid X = 4 and tid Z = 1
Block 0 -> tid X = 0 and tid Z = 2
Block 0 -> tid X = 1 and tid Z = 2
Block 0 -> tid X = 2 and tid Z = 2
Block 0 -> tid X = 3 and tid Z = 2
Block 0 -> tid X = 4 and tid Z = 2
```

b. Modify the code again so that it creates a 3D block instead of 2D block. Also print thread ids along its corresponding directions/axis.

hpc_herald - 3.cu

```cpp
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void display() {
    printf("Block %d -> tid X = %d and Y = %d and tid Z = %d\n",
        blockIdx.x, threadIdx.x, threadIdx.y, threadIdx.z);
}
int main() {
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(3, 2, 3);
    display << <gridSize, blockSize >> > ();
    cudaDeviceSynchronize();
    return 0;
}
```

[25] ✓ 1s

```
1 !nvcc -arch=sm_75 -o 3d_threads 3d_threads.cu
```

[26] ✓ 0s

```
1 !./3d_threads
```

```
Block 0 -> tid X = 0 and Y = 0 and tid Z = 0
Block 0 -> tid X = 1 and Y = 0 and tid Z = 0
Block 0 -> tid X = 2 and Y = 0 and tid Z = 0
Block 0 -> tid X = 0 and Y = 1 and tid Z = 0
Block 0 -> tid X = 1 and Y = 1 and tid Z = 0
Block 0 -> tid X = 2 and Y = 1 and tid Z = 0
Block 0 -> tid X = 0 and Y = 0 and tid Z = 1
Block 0 -> tid X = 1 and Y = 0 and tid Z = 1
Block 0 -> tid X = 2 and Y = 0 and tid Z = 1
Block 0 -> tid X = 0 and Y = 1 and tid Z = 1
Block 0 -> tid X = 1 and Y = 1 and tid Z = 1
Block 0 -> tid X = 2 and Y = 1 and tid Z = 1
Block 0 -> tid X = 0 and Y = 0 and tid Z = 2
Block 0 -> tid X = 1 and Y = 0 and tid Z = 2
Block 0 -> tid X = 2 and Y = 0 and tid Z = 2
Block 0 -> tid X = 0 and Y = 1 and tid Z = 2
Block 0 -> tid X = 1 and Y = 1 and tid Z = 2
Block 0 -> tid X = 2 and Y = 1 and tid Z = 2
```

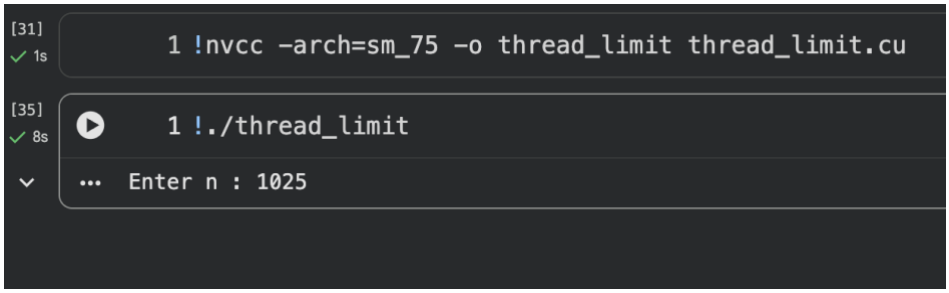
Question 4: Printing Thread Indices Using CUDA

The following code takes an integer n from the user and prints all numbers from 0 to n - 1 using a CUDA kernel.

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void displayIdx(){
    int tid = threadIdx.x;
    printf("%d\n", tid);
}

int main(){
    int n;
    printf("Enter n : ");
    scanf("%d", &n);
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(n, 1, 1);
    displayIdx<<<gridSize, blockSize>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

a. Does the code work when the user enters n as 1025? Explain.

No, the code doesn't work when user enters n as 1025.



This is because, there is a limit on number of threads that can be used within a single block and that is 1024. As the above execution attempts to create 1025 thread within a block, the GPU terminates the program with runtime error of invalid configuration.

b. Does the code still work when changing the dim3 blockSize(n, 1, 1) line of code to dim3 blockSize(n, n, 1)? Modify the code so that the values are correctly printed out.

hpc_herald - 4.cu

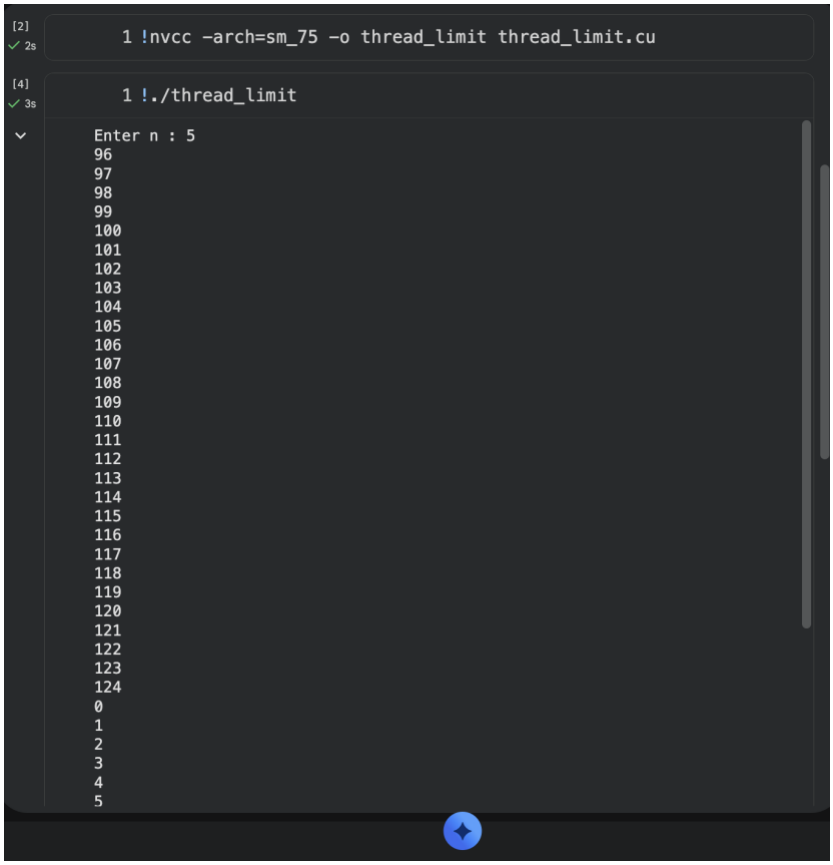
```cu
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void displayIdx() {
    int tid = blockDim.x * threadIdx.y + threadIdx.x;
    printf("%d\n", tid);
}
int main() {
    int n;
    printf("Enter n : ");
    scanf("%d", &n);
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(n, n, 1);
    displayIdx << <gridSize, blockSize >> > ();
    cudaDeviceSynchronize();
    return 0;
}
```

```
[9] ✓ 1s
1 !nvcc -arch=sm_75 -o thread_limit thread_limit.cu

[11] ✓ 5s
1 !./thread_limit

Enter n : 5
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
```

c. **Now change the dim3 blockSize(n, n, 1) to dim3 blockSize(n, n, n) and also modify the code so that it correctly prints out the value. Example if n = 5, the code should print the value 0 to 124.**

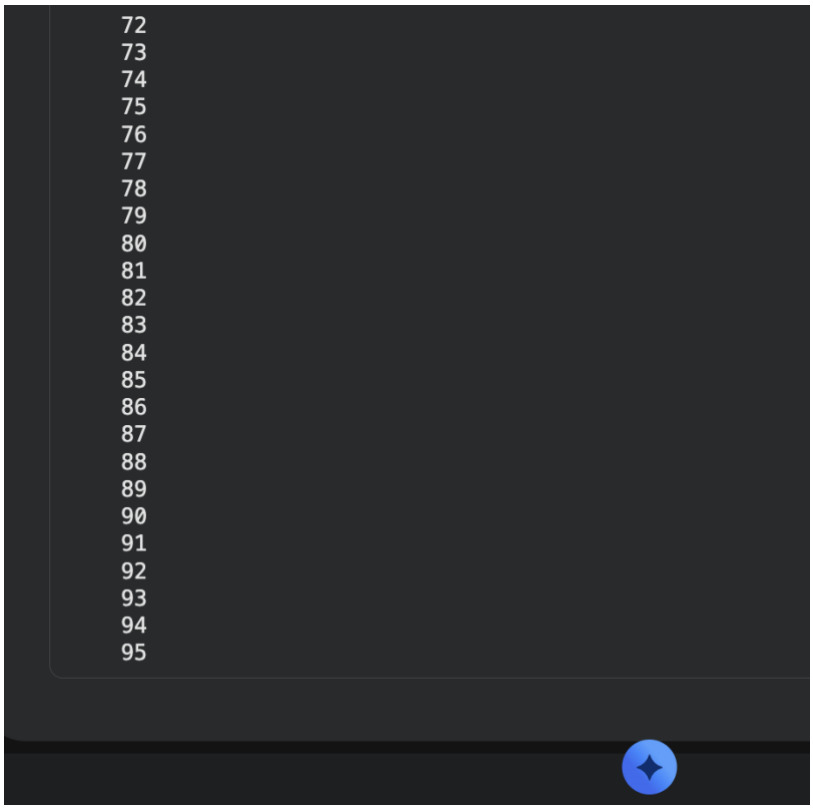
hpc_herald - 4.cu

```cpp
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void displayIdx() {
    int tid = threadIdx.z * blockDim.x * blockDim.y
        + threadIdx.y * blockDim.x
        + threadIdx.x;
    printf("%d\n", tid);
}
int main() {
    int n;
    printf("Enter n : ");
    scanf("%d", &n);
    dim3 gridSize(1, 1, 1);
    dim3 blockSize(n, n, n);
    displayIdx << <gridSize, blockSize >> > ();
    cudaDeviceSynchronize();
    return 0;
}
```


[2]
2s
1 !nvcc -arch=sm_75 -o thread_limit thread_limit.cu
[4]
3s
1 !./thread_limit
Enter n : 5
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
0
1
2
3
4
5


[2]
2s
1 !nvcc -arch=sm_75 -o thread_limit thread_limit.cu
[4]
3s
1 !./thread_limit
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37


[2]
2s
1 !nvcc -arch=sm_75 -o thread_limit thread_limit.cu
[4]
3s
1 !./thread_limit
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73


72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95

Question 5: Quadratic Solution

Create a CUDA program to solve quadratics (find the roots). You will be given a text file containing 3 integers on each row which represent a, b and c for the following formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Explanation:

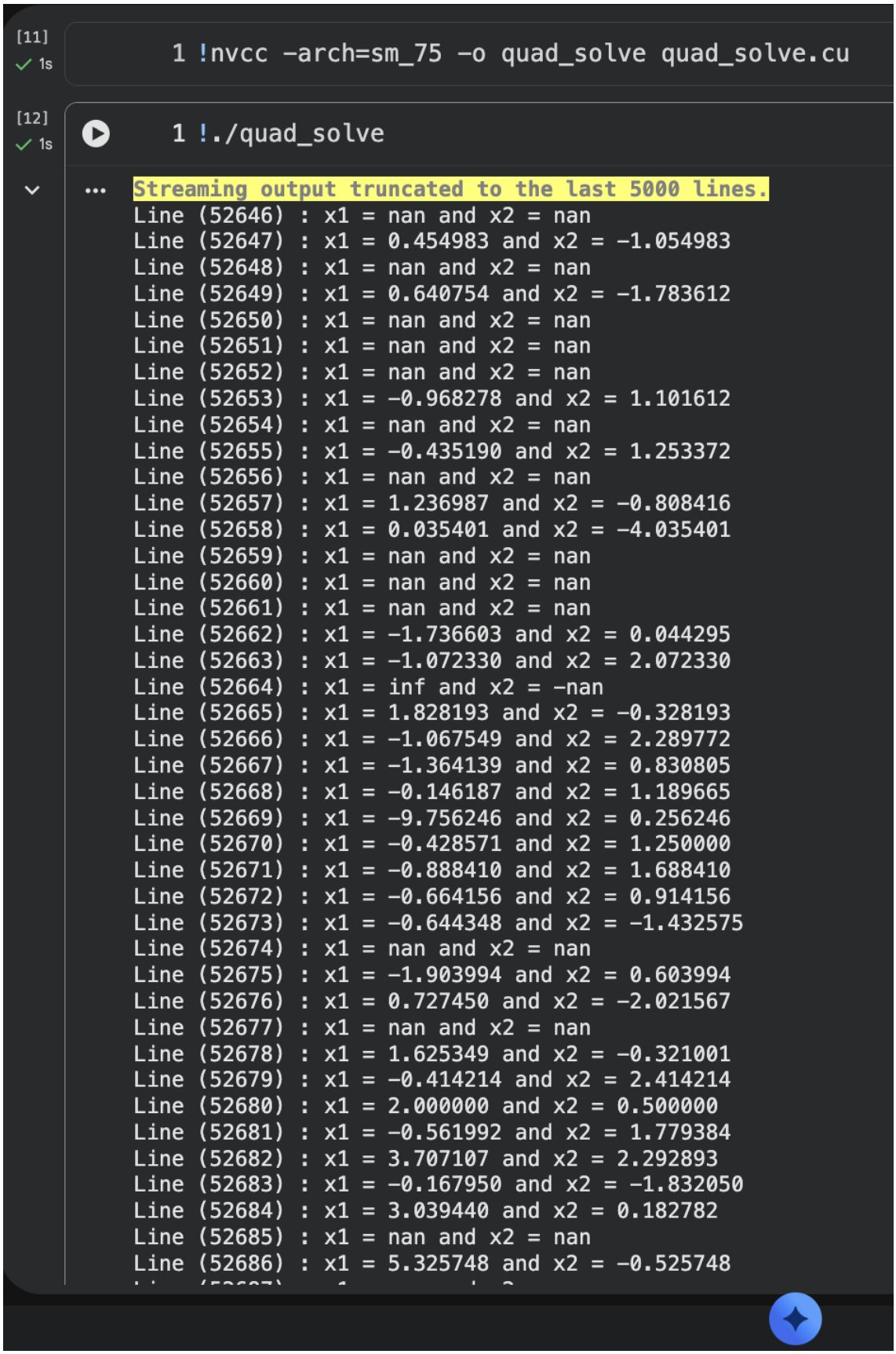
First, the program reads the file "QuadData.txt" containing sets of three numbers (a, b, c) that represent different equations. It counts how many equations there are, allocates memory on the host to store them, and loads the data.

Next, it allocates memory on the GPU and copies the equation data there so the GPU works on it. The program then launches the kernel, in which each equation is assigned to a separate thread, meaning many equations are solved simultaneously instead of one after another.

The program calculates the required number of blocks based on the total number of equations and a predefined THREADS_PER_BLOCK constant (set to 256). If the blocks isn't a perfect division, then, one extra block is added to ensure enough threads. In terms of thread organization, each block contains 256 threads structured in a one-dimensional arrangement, while the grid itself comprises multiple blocks also arranged in a one-dimensional row. Within this hierarchy, every thread is assigned a unique identifier, calculated as the block ID multiplied by the block size, plus the thread's position within its specific block. This systematic numbering allows for efficient work distribution. Inside the kernel, each thread first verifies whether its computed ID corresponds to a valid equation number. If a thread's ID exceeds the total number of equations, it exits immediately. This safeguard prevents any superfluous threads from performing unnecessary computational work.

Each thread calculates the two possible solutions for its equation using the quadratic formula. If the equation has no real solutions, the result store NAN. Once all GPU threads finish, the solutions are copied back from the GPU to the computer's memory. The program then prints each equation's solutions to the screen. Finally, all memory used on both the computer and the GPU are freed.

Output:


[11]
1s
1 !nvcc -arch=sm_75 -o quad_solve quad_solve.cu
[12]
1s
1 !./quad_solve
Streaming output truncated to the last 5000 lines.
Line (52646) : x1 = nan and x2 = nan
Line (52647) : x1 = 0.454983 and x2 = -1.054983
Line (52648) : x1 = nan and x2 = nan
Line (52649) : x1 = 0.640754 and x2 = -1.783612
Line (52650) : x1 = nan and x2 = nan
Line (52651) : x1 = nan and x2 = nan
Line (52652) : x1 = nan and x2 = nan
Line (52653) : x1 = -0.968278 and x2 = 1.101612
Line (52654) : x1 = nan and x2 = nan
Line (52655) : x1 = -0.435190 and x2 = 1.253372
Line (52656) : x1 = nan and x2 = nan
Line (52657) : x1 = 1.236987 and x2 = -0.808416
Line (52658) : x1 = 0.035401 and x2 = -4.035401
Line (52659) : x1 = nan and x2 = nan
Line (52660) : x1 = nan and x2 = nan
Line (52661) : x1 = nan and x2 = nan
Line (52662) : x1 = -1.736603 and x2 = 0.044295
Line (52663) : x1 = -1.072330 and x2 = 2.072330
Line (52664) : x1 = inf and x2 = -nan
Line (52665) : x1 = 1.828193 and x2 = -0.328193
Line (52666) : x1 = -1.067549 and x2 = 2.289772
Line (52667) : x1 = -1.364139 and x2 = 0.830805
Line (52668) : x1 = -0.146187 and x2 = 1.189665
Line (52669) : x1 = -9.756246 and x2 = 0.256246
Line (52670) : x1 = -0.428571 and x2 = 1.250000
Line (52671) : x1 = -0.888410 and x2 = 1.688410
Line (52672) : x1 = -0.664156 and x2 = 0.914156
Line (52673) : x1 = -0.644348 and x2 = -1.432575
Line (52674) : x1 = nan and x2 = nan
Line (52675) : x1 = -1.903994 and x2 = 0.603994
Line (52676) : x1 = 0.727450 and x2 = -2.021567
Line (52677) : x1 = nan and x2 = nan
Line (52678) : x1 = 1.625349 and x2 = -0.321001
Line (52679) : x1 = -0.414214 and x2 = 2.414214
Line (52680) : x1 = 2.000000 and x2 = 0.500000
Line (52681) : x1 = -0.561992 and x2 = 1.779384
Line (52682) : x1 = 3.707107 and x2 = 2.292893
Line (52683) : x1 = -0.167950 and x2 = -1.832050
Line (52684) : x1 = 3.039440 and x2 = 0.182782
Line (52685) : x1 = nan and x2 = nan
Line (52686) : x1 = 5.325748 and x2 = -0.525748

```
Line (57599) : x1 = -0.257309 and x2 = 0.971595
Line (57600) : x1 = 0.356963 and x2 = -2.801408
Line (57601) : x1 = nan and x2 = nan
Line (57602) : x1 = -0.247722 and x2 = 0.695998
Line (57603) : x1 = 5.791288 and x2 = 1.208712
Line (57604) : x1 = nan and x2 = nan
Line (57605) : x1 = nan and x2 = nan
Line (57606) : x1 = -0.500000 and x2 = 12.000000
Line (57607) : x1 = 0.780945 and x2 = -0.521685
Line (57608) : x1 = nan and x2 = nan
Line (57609) : x1 = nan and x2 = nan
Line (57610) : x1 = nan and x2 = nan
Line (57611) : x1 = nan and x2 = nan
Line (57612) : x1 = 1.439337 and x2 = -0.972670
Line (57613) : x1 = -0.600000 and x2 = -0.500000
Line (57614) : x1 = nan and x2 = nan
Line (57615) : x1 = -12.830952 and x2 = -1.169048
Line (57616) : x1 = 13.844289 and x2 = 1.155711
Line (57617) : x1 = -0.471780 and x2 = 1.271780
Line (57618) : x1 = nan and x2 = nan
Line (57619) : x1 = -nan and x2 = -inf
Line (57620) : x1 = -1.828427 and x2 = 3.828427
Line (57621) : x1 = nan and x2 = nan
Line (57622) : x1 = -0.380387 and x2 = 0.725214
Line (57623) : x1 = -0.122499 and x2 = 6.122499
Line (57624) : x1 = 0.243423 and x2 = 28.756577
Line (57625) : x1 = nan and x2 = nan
Line (57626) : x1 = nan and x2 = nan
Line (57627) : x1 = 1.341641 and x2 = -1.341641
Line (57628) : x1 = 0.965393 and x2 = -8.632060
Line (57629) : x1 = -0.272110 and x2 = 1.272110
Line (57630) : x1 = 1.586607 and x2 = -1.386607
Line (57631) : x1 = 1.580764 and x2 = -1.119225
Line (57632) : x1 = nan and x2 = nan
Line (57633) : x1 = -0.783966 and x2 = 0.610053
Line (57634) : x1 = -0.291366 and x2 = 5.491366
Line (57635) : x1 = -1.711841 and x2 = 0.922368
Line (57636) : x1 = nan and x2 = nan
Line (57637) : x1 = -2.623774 and x2 = -0.076226
Line (57638) : x1 = -2.750000 and x2 = 1.000000
Line (57639) : x1 = -1.000000 and x2 = -1.125000
Line (57640) : x1 = nan and x2 = nan
Line (57641) : x1 = -2.222586 and x2 = 0.858950
Line (57642) : x1 = -0.517385 and x2 = 1.417385
Line (57643) : x1 = -0.298991 and x2 = -1.951009
Line (57644) : x1 = nan and x2 = nan
Line (57645) : x1 = nan and x2 = nan
```

hpc_herald - 5.cu

```cuda
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define THREADS_PER_BLOCK 256  // Number of threads per CUDA block

/**
 * Counts the number of lines in a file
 * @param fptr - Pointer to the FILE object
 * @return Number of lines in the file
 */
int countLines(FILE* fptr);

/**
 * Reads quadratic equation coefficients from file into memory
 * @param fptr - Pointer to the FILE object
 * @param a - Array to store 'a' coefficients
 * @param b - Array to store 'b' coefficients
 * @param c - Array to store 'c' coefficients
 * @param number_of_lines - Number of lines/equations to read
 */
void readFileIntoMemory(FILE* fptr, int* a, int* b, int* c, int number_of_lines);

/**
 * Deallocates host memory for coefficient arrays
 * @param h_a - Host array for 'a' coefficients
 * @param h_b - Host array for 'b' coefficients
 * @param h_c - Host array for 'c' coefficients
 */
void deallocateHostMemory(int* h_a, int* h_b, int* h_c);

/**
 * Deallocates device memory for coefficient arrays
 * @param d_a - Device array for 'a' coefficients
 * @param d_b - Device array for 'b' coefficients
 * @param d_c - Device array for 'c' coefficients
 */
void deallocateDeviceMemory(int* d_a, int* d_b, int* d_c);

/**
 * Deallocates memory for solution arrays (both host and device)
 * @param h_x1 - Host array for first solutions (x1)
 * @param h_x2 - Host array for second solutions (x2)
 * @param d_x1 - Device array for first solutions (x1)
 * @param d_x2 - Device array for second solutions (x2)
 */
void deallocateResultMemory(double* h_x1, double* h_x2, double* d_x1, double* d_x2);

```

hpc_herald - 5.cu

```cuda
/**
 * CUDA kernel function to solve quadratic equations in parallel
 * Each thread solves one quadratic equation: ax² + bx + c = 0
 * @param d_a - Device array of 'a' coefficients
 * @param d_b - Device array of 'b' coefficients
 * @param d_c - Device array of 'c' coefficients
 * @param d_x1 - Device array to store first solutions
 * @param d_x2 - Device array to store second solutions
 * @param number_of_lines - Total number of equations to solve
 */
__global__ void solve_quad(int* d_a, int* d_b, int* d_c, double* d_x1, double* d_x2, int number_of_lines);

int main() {
    int lines;  // Number of equations/lines in the file

    // Host (CPU) memory pointers for coefficients
    int* h_a, * h_b, * h_c;

    // Device (GPU) memory pointers for coefficients
    int* d_a, * d_b, * d_c;

    // Host and device memory pointers for solutions
    double* h_x1, * h_x2;
    double* d_x1, * d_x2;

    // 1. Open file to read data
    FILE* fptr = fopen("QuadData.txt", "r");
    if (fptr == NULL) {
        perror("File open failed");
        return 1;
    }

    // 2. Count number of lines in the file
    lines = countLines(fptr);

    // 3. Allocate host memory to store the file data
    h_a = (int*)malloc(lines * sizeof(int));
    h_b = (int*)malloc(lines * sizeof(int));
    h_c = (int*)malloc(lines * sizeof(int));

    // 4. Read the data from file to host memory
    readFileIntoMemory(fptr, h_a, h_b, h_c, lines);

    // 5. Close the file
    fclose(fptr); // File Closed

```

hpc_herald - 5.cu

```
    // 6. Allocate device memory for coefficients and solutions
    cudaMalloc(&d_a, lines * sizeof(int));
    cudaMalloc(&d_b, lines * sizeof(int));
    cudaMalloc(&d_c, lines * sizeof(int));
    cudaMalloc(&d_x1, lines * sizeof(double));
    cudaMalloc(&d_x2, lines * sizeof(double));

    // 7. Copy coefficients from host to device memory
    cudaMemcpy(d_a, h_a, lines * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, lines * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_c, h_c, lines * sizeof(int), cudaMemcpyHostToDevice);

    // 8. Launch kernel routine to solve equations in parallel
    // Calculate block size based on number of equations and threads per block
    int num_blocks = lines / THREADS_PER_BLOCK;
    if (lines % THREADS_PER_BLOCK > 0)
        num_blocks++;  // Add extra block if lines don't divide evenly

    // Define grid and block dimensions
    dim3 gd(num_blocks, 1, 1);  // Grid dimensions
    dim3 bd(THREADS_PER_BLOCK, 1, 1);  // Block dimensions

    // Launch the CUDA kernel
    solve_quad << <gd, bd >> > (d_a, d_b, d_c, d_x1, d_x2, lines);
    cudaDeviceSynchronize();  // Wait for kernel to complete

    // 9. Allocate host memory to store results
    h_x1 = (double*)malloc(lines * sizeof(double));
    h_x2 = (double*)malloc(lines * sizeof(double));

    // 10. Copy results from device to host
    cudaMemcpy(h_x1, d_x1, lines * sizeof(double), cudaMemcpyDeviceToHost);
    cudaMemcpy(h_x2, d_x2, lines * sizeof(double), cudaMemcpyDeviceToHost);

    // 11. Print the solutions
    for (int i = 0; i < lines; i++) {
        printf("Line (%d) : x1 = %lf and x2 = %lf\n", i + 1, h_x1[i], h_x2[i]);
    }

    // 12. Free all allocated memory
    deallocateHostMemory(h_a, h_b, h_c);
    deallocateDeviceMemory(d_a, d_b, d_c);
    deallocateResultMemory(h_x1, h_x2, d_x1, d_x2);

    return 0;
}
```

hpc_herald - 5.cu

```
141
142 int countLines(FILE* fptr) {
143     int number_of_lines = 0;
144     int ch, prev = '\n';
145
146     rewind(fptr);  // Reset file pointer to beginning
147
148     // Count newline characters
149     while ((ch = fgetc(fptr)) != EOF) {
150         if (ch == '\n')
151             number_of_lines++;
152     }
153
154     // Handle case where file doesn't end with newline
155     if (prev != '\n') number_of_lines++;
156
157     rewind(fptr);  // Reset file pointer for subsequent reading
158     return number_of_lines;
159 }
160
161 void readFileIntoMemory(FILE* fptr, int* a, int* b, int* c, int number_of_lines) {
162     rewind(fptr);  // Ensure we're at the beginning of the file
163
164     // Read coefficients in format: a,b,c (comma-separated)
165     for (int i = 0; i < number_of_lines; i++) {
166         if (fscanf(fptr, "%d,%d,%d", &a[i], &b[i], &c[i]) != 3) {
167             printf("Invalid data at line %d\n", i + 1);
168             deallocateHostMemory(a, b, c);
169             exit(1);
170         }
171     }
172 }
173
174 void deallocateHostMemory(int* h_a, int* h_b, int* h_c) {
175     free(h_a);
176     free(h_b);
177     free(h_c);
178 }
179
180 void deallocateDeviceMemory(int* d_a, int* d_b, int* d_c) {
181     cudaFree(d_a);
182     cudaFree(d_b);
183     cudaFree(d_c);
184 }
```

hpc_herald - 5.cu

```cuda

void deallocateResultMemory(double* h_x1, double* h_x2, double* d_x1, double* d_x2) {
    cudaFree(d_x1);  // Free device memory for x1 solutions
    cudaFree(d_x2);  // Free device memory for x2 solutions
    free(h_x1);      // Free host memory for x1 solutions
    free(h_x2);      // Free host memory for x2 solutions
}

__global__ void solve_quad(int* d_a, int* d_b, int* d_c, double* d_x1, double* d_x2, int number_of_lines) {
    // Calculate thread ID - each thread handles one equation
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    // Check if thread ID is within bounds of number of equations
    if (tid >= number_of_lines) return;

    double a, b, c;
    double x1 = 0.0, x2 = 0.0;

    // Get coefficients for this thread's equation
    a = (double)d_a[tid];
    b = (double)d_b[tid];
    c = (double)d_c[tid];

    // Calculate discriminant: b² - 4ac
    double discriminant = (b * b) - 4.0 * a * c;

    // Check if solutions are real
    if (discriminant < 0) {
        // No real solutions - store NaN (Not a Number)
        x1 = NAN;
        x2 = NAN;
    } else {
        // Real solutions exist - calculate using quadratic formula
        double square_root = sqrt(discriminant);
        double twice_a = 2 * a;
        x1 = (-b + square_root) / twice_a;  // First root
        x2 = (-b - square_root) / twice_a;  // Second root
    }

    // Store solutions in device memory
    d_x1[tid] = x1;
    d_x2[tid] = x2;
}

```